Authentifizierung & Autorisierung

Lernunterlage zum Projekt Home Control

JWT · Access Token · Refresh Token · Refresh-Token-Rotation · Passwort-Hashing · Rollen (RBAC) · XSS · CSRF · HTTP-only Cookies · 401 und 403

Stand: 29.10.2026 · Branch `feature/authentication` · ASP.NET Core 10 Minimal API, EF Core, SQLite, React

Der wichtigste Satz vorweg

Authentifizierung beantwortet *„Wer bist du?“*, **Autorisierung** beantwortet *„Was darfst du?“*. Authentifizierung kommt immer zuerst: Erst wenn der Server weiß, wer anfragt, kann er entscheiden, was diese Person darf.

Inhalt

1 Grundbegriffe und Statuscodes 401 / 403
2 Tokenbasierte Anmeldung: Access Token und Refresh Token
3 JWT: Aufbau, Signatur, Prüfung
4 Refresh-Token-Rotation und Reuse Detection
5 Passwort-Hashing (und warum drei Geheimnisse drei Verfahren brauchen)
6 Rollen (RBAC) in Home Control
7 Speicherort im Browser, XSS, CSRF und Cookie-Optionen
8 Der komplette Ablauf in Home Control
9 Der Code: Dateien, Endpunkte, Middleware, Konfiguration, Tests
10 Bekannte Grenzen
11 Kurzantworten für Rückfragen in der Präsentation
12 Merksätze
Anhang: Infografiken

1 Grundbegriffe und Statuscodes

	Authentifizierung	Autorisierung
Frage	Wer bist du?	Was darfst du?
Beispiel im Alltag	Personalausweis am Flughafen vorzeigen	Bordkarte: welches Flugzeug, welcher Sitz
In Home Control	E-Mail und Passwort prüfen, danach Access Token prüfen	Rolle im Token prüfen: Admin oder User
ASP.NET Core	`UseAuthentication()`, JWT-Bearer-Middleware	`UseAuthorization()`, `RequireAuthorization()`
Fehlt es / schlägt es fehl	**401 Unauthorized**	**403 Forbidden**

401 oder 403?

Der englische Name von 401 („Unauthorized“) ist historisch irreführend. Gemeint ist „nicht authentifiziert“.

Status	Bedeutung	Typische Ursache	Was der Client tun sollte
401	Nicht (korrekt) angemeldet	Kein Token, Token abgelaufen, Signatur falsch, Passwort falsch	Token erneuern (Refresh) oder neu einloggen
403	Angemeldet, aber nicht berechtigt	Ein User ruft eine Admin-Funktion auf	Nicht erneut versuchen, ein neues Token ändert nichts an der Rolle

> **Merksatz:** 401 = „Ich weiß nicht, wer du bist.“ 403 = „Ich weiß, wer du bist, aber du darfst das nicht.“

2 Tokenbasierte Anmeldung

Früher hat sich der Server nach dem Login meist eine **Session** gemerkt (Sitzungs-Id im Cookie, Daten im Arbeitsspeicher des Servers). Bei **tokenbasierter** Anmeldung bekommt der Client nach dem Login ein Token und schickt es bei jeder Anfrage mit. Der Server erkennt am Token selbst, wer anfragt.
Home Control nutzt zwei Tokens mit unterschiedlichen Aufgaben:

	Access Token	Refresh Token
Wofür?	Zugriff auf geschützte API-Endpunkte	Ein neues Access Token holen, ohne Passwort
Format	JWT (signiert, lesbarer Inhalt)	64 zufällige Bytes, ohne Inhalt
Laufzeit	15 Minuten	30 Tage
Wird gesendet	Bei jeder API-Anfrage im Header `Authorization: Bearer <token>`	Nur an `/api/auth/refresh` und `/logout`, automatisch per Cookie
Gespeichert im Browser	Local Storage	HTTP-only Cookie `hc_refresh`
Gespeichert auf dem Server	Gar nicht. Der Server prüft nur die Signatur.	Nur der SHA-256-Hash, in der Tabelle RefreshTokens

	Access Token	Refresh Token
Widerrufbar?	Nein, läuft einfach ab	Ja, jederzeit (RevokedAtUtc)

Warum zwei Tokens? Ein kurzes Access Token begrenzt den Schaden, wenn es gestohlen wird: Nach spätestens 15 Minuten ist es wertlos. Damit sich der Benutzer trotzdem nicht alle 15 Minuten neu einloggen muss, gibt es das langlebige Refresh Token. Das ist besser geschützt (HTTP-only Cookie, nur Hash in der Datenbank) und lässt sich widerrufen.

3 JWT (JSON Web Token)

Ein JWT besteht aus drei Teilen, jeweils Base64Url-kodiert und durch Punkte getrennt:

```
eyJhbGciOiJIUzI1NiIs...  .  eyJzdWIiOiIxMTEx...  .  q8Vt2bX0sF...
└──── Header ─────┘        └─── Payload ──┘        └─ Signatur ─┘
```

Teil	Inhalt	Beispiel
Header	Verfahren der Signatur	`{"alg":"HS256","typ":"JWT"}`
Payload	Claims: Angaben über den Benutzer und die Gültigkeit	siehe unten
Signatur	HMAC-SHA256 über Header und Payload, mit dem geheimen Schlüssel des Servers	nicht lesbar

So sieht der Payload eines Home-Control-Tokens aus:

```
{
  "sub":   "11111111-2222-3333-4444-555555555555",  // Benutzer-Id
  "email": "mario@example.de",
  "role":  "Admin",
  "iss":   "HomeControl",    // Aussteller (Issuer)
  "aud":   "HomeControl",    // Empfänger (Audience)
  "iat":   1793260800,       // ausgestellt um (Unix-Zeit)
  "nbf":   1793260800,       // gültig ab
  "exp":   1793261700        // gültig bis = iat + 15 Minuten
}
```

Signiert heißt nicht verschlüsselt. Jeder, der ein JWT hat, kann den Payload lesen (z. B. auf jwt.io). Deshalb stehen darin nur Id, E-Mail und Rolle. Niemals Passwörter, Passwort-Hashes oder Zugangsdaten zu Tuya oder Blink.

Was die Signatur bringt

Ändert jemand den Payload, zum Beispiel `"role":"User"` in `"role":"Admin"`, passt die Signatur nicht mehr. Eine neue, passende Signatur kann nur erzeugen, wer den geheimen Schlüssel kennt. Der Test `Veraendertes_Token_wird_abgelehnt` zeigt genau diesen Angriff.

Was der Server bei jeder Anfrage prüft

- **Signatur** mit dem geheimen Schlüssel (`Jwt:SigningKey`)
- **Issuer** und **Audience**: Token stammt von Home Control und ist für Home Control gedacht
- **Ablaufzeit** (`exp`, `nbf`) mit 30 Sekunden Toleranz (ClockSkew). Der Standard wären 5 Minuten, dann gälte ein 15-Minuten-Token faktisch 20 Minuten.

HS256 heißt: Derselbe geheime Schlüssel signiert und prüft. Das passt, weil nur unser eigener Server Tokens ausstellt und prüft. (Bei RS256 gäbe es ein Schlüsselpaar: privat signieren, öffentlich

prüfen. Das braucht man, wenn fremde Dienste Tokens prüfen sollen.)

4 Refresh-Token-Rotation und Reuse Detection

Rotation bedeutet: Jedes Refresh Token ist nur **einmal** benutzbar. Bei jedem Refresh bekommt der Client ein neues, das alte wird ungültig.

Ablauf von POST /api/auth/refresh

Schritt	Was passiert
1	Browser schickt automatisch das Cookie `hc_refresh` mit.
2	Server berechnet den SHA-256-Hash des Tokens und sucht ihn in der Tabelle RefreshTokens.
3	Nicht gefunden oder abgelaufen: **401**, Cookie wird gelöscht.
4	Schon widerrufen: **Reuse Detection** (siehe unten), **401**.
5	Gültig: altes Token bekommt `RevokedAtUtc = jetzt`.
6	Neues Access Token und neues Refresh Token werden erzeugt, der Hash des neuen wird gespeichert.
7	Antwort: neues Access Token im JSON, neues Refresh Token im Cookie.

Reuse Detection: warum widerrufen statt löschen
Nach einer Rotation hat der echte Client immer schon das *neue* Token. Taucht das *alte* noch einmal auf, hat es sehr wahrscheinlich jemand kopiert. Der Server weiß nicht, wer von beiden der echte Benutzer ist, und beendet deshalb **alle** Sitzungen dieses Benutzers. Beide müssen sich neu einloggen, und der Dieb kann das nicht, weil er das Passwort nicht kennt.
Das funktioniert nur, weil alte Tokens als widerrufen **markiert** und nicht gelöscht werden. Ein gelöschtes Token wäre beim zweiten Versuch einfach „unbekannt“, der Angriff bliebe unbemerkt.

Logout

`POST /api/auth/logout` widerruft das Refresh Token und löscht das Cookie. Das Access Token kann der Server nicht zurückholen, weil er es sich nie gemerkt hat. Das Frontend löscht es aus dem Local Storage, und spätestens nach 15 Minuten läuft es ohnehin ab.

5 Passwort-Hashing

	Hashing	Verschlüsselung
Richtung	Nur eine Richtung: aus dem Hash kommt man nicht zurück	Zwei Richtungen: mit dem Schlüssel wieder lesbar
Prüfen	Eingabe erneut hashen und Hashes vergleichen	Entschlüsseln und vergleichen
Für Passwörter?	**Ja**	Nein. Wer den Schlüssel hat, hätte alle Passwörter.

Ein Passwort-Hash muss zwei Dinge können:

- **Salt**: eine Zufallszahl pro Benutzer, die mitgehasht wird. Gleiche Passwörter ergeben dadurch verschiedene Hashes, und vorberechnete Tabellen (Rainbow Tables) nützen nichts.
- **Langsam sein**: viele tausend Durchläufe. Ein Login merkt das kaum, ein Angreifer, der Milliarden Passwörter durchprobieren will, sehr wohl. Bekannte Verfahren: PBKDF2, bcrypt, Argon2.

Home Control nutzt den `PasswordHasher<User>` aus ASP.NET Core Identity (PBKDF2 mit HMAC-SHA512, Salt und 100.000 Durchläufen). Der gespeicherte Text enthält Verfahren, Salt und Hash zusammen. Ist der Hash mit einem älteren Verfahren entstanden, meldet `VerifyHashedPassword` `SuccessRehashNeeded`, und Home Control hasht beim Login neu.

Drei Geheimnisse, drei Verfahren

Geheimnis	Gespeichert als	Warum so?
Passwort des Benutzers	Langsamer Hash mit Salt (PBKDF2)	Von Menschen gewählt, oft kurz und erratbar. Muss nie wieder im Klartext gebraucht werden.
Home-Control-Refresh-Token	Schneller Hash (SHA-256), ohne Salt	Zufallswert mit 512 Bit, nicht erratbar. Ein schneller Hash reicht, Salt bringt nichts.
Blink-Refresh-Token	Verschlüsselt (ASP.NET Data Protection)	Wir müssen es selbst wieder an Blink schicken, also brauchen wir den Klartext zurück.

6 Rollen (RBAC)

RBAC heißt *Role-Based Access Control*: Rechte hängen an der Rolle, nicht an einzelnen Personen. Die Rolle steht im JWT (Claim `role`) und wird bei jeder Anfrage aus dem Token gelesen.

Rolle	Darf (Plan)	Stand jetzt
Admin	Räume, Geräte und Benutzer verwalten, sensible Blink-Funktionen, Reports	Kann weitere Benutzer anlegen (POST /api/auth/register)
User	Räume, Geräte, Gerätestatus, Ereignisse und Reports ansehen	Kann sich anmelden und das eigene Profil sehen

Wer wird Admin?

- Gibt es noch **keinen** Benutzer, darf sich jeder registrieren und wird automatisch **Admin**. So kommt man beim ersten Start an ein Admin-Konto.
- Danach darf **nur ein Admin** neue Benutzer anlegen (Standardrolle User). Ohne Login: 401. Als User angemeldet: 403.

Die bestehenden Endpunkte werden schrittweise geschützt. Bisher ist nur `/api/auth/profile` gesperrt, damit Frontend und der n8n-Agent bis zum Frontend-Login weiter funktionieren. Geplant ist: Lesen für alle Angemeldeten, Anlegen, Ändern und Löschen nur für Admin (`RequireAuthorization(policy => policy.RequireRole("Admin"))`).

7 Speicherort im Browser, XSS und CSRF

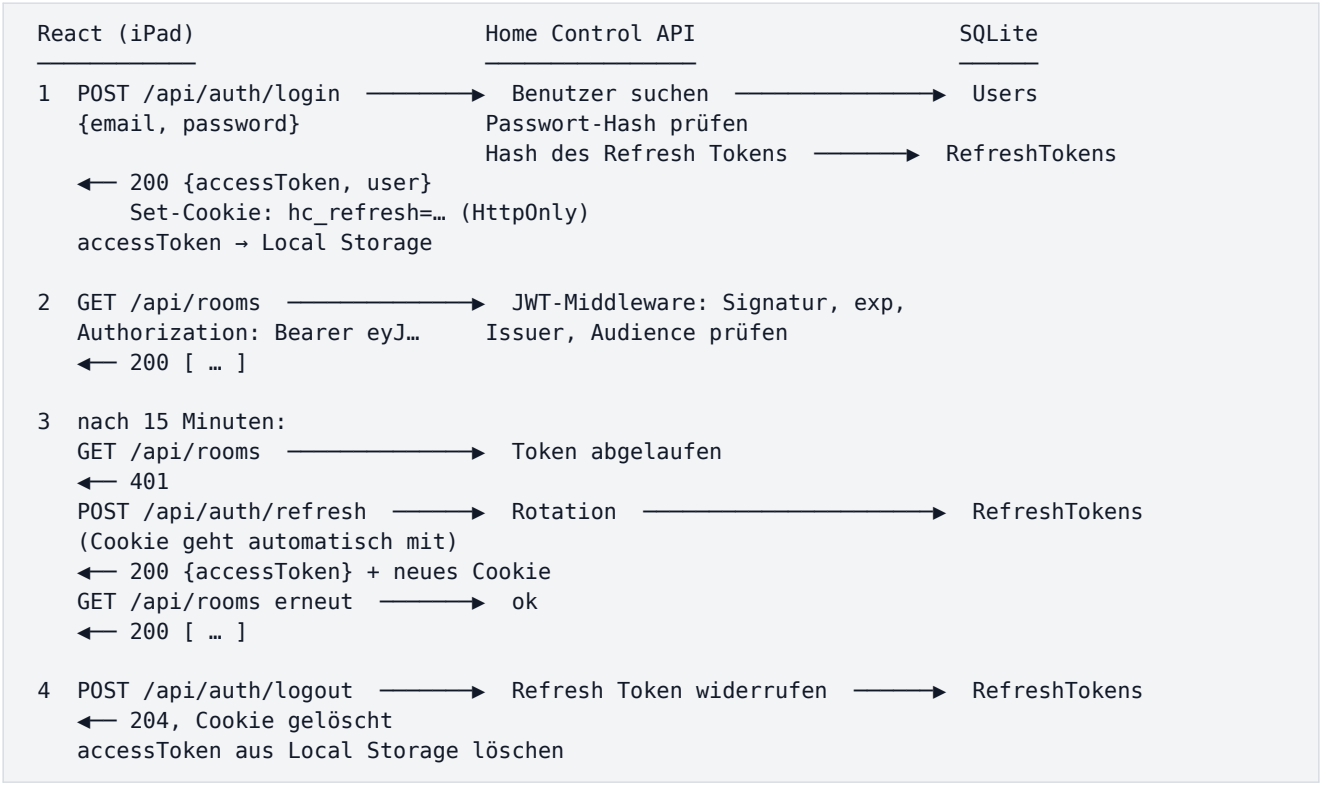
Angriff	Was passiert	Wogegen er hilft / Schutz in Home Control
XSS Cross-Site Scripting	Fremdes JavaScript läuft auf unserer Seite (z. B. über einen ungefilterten Gerätenamen) und kann alles lesen, was JavaScript lesen kann, also auch den Local Storage.	Refresh Token im **HTTP-only Cookie**: für JavaScript unsichtbar. React maskiert Text in JSX automatisch, kein `dangerouslySetInnerHTML`. Das Access Token im Local Storage bleibt angreifbar, gilt aber nur 15 Minuten.
CSRF Cross-Site Request Forgery	Eine fremde Webseite lässt den Browser eine Anfrage an unsere API schicken. Der Browser hängt Cookies automatisch an.	**SameSite=Strict**: Der Browser schickt das Cookie bei Anfragen von fremden Seiten nicht mit. Das Access Token steckt im Header, den eine fremde Seite nicht setzen kann.

Die Optionen des Refresh-Cookies

Option	Wert	Wirkung
HttpOnly	true	JavaScript kann das Cookie nicht lesen (Schutz bei XSS).
Secure	true außer in Development	Cookie nur über HTTPS. Lokal läuft http://localhost, sonst ginge der Login gar nicht.
SameSite	Strict	Kein Mitsenden bei Anfragen, die von fremden Seiten ausgelöst werden (Schutz vor CSRF).
Path	/api/auth	Cookie geht nur an die Auth-Endpunkte, nicht bei jedem Abruf von Räumen oder Geräten.
Expires	30 Tage	Gleiche Laufzeit wie das Refresh Token in der Datenbank.

Warum nicht beide Tokens im Cookie? Das wäre gegen XSS noch sicherer, aber dann müsste jede API-Anfrage per Cookie authentifiziert und gegen CSRF geschützt werden (z. B. mit einem zusätzlichen CSRF-Token). Die Aufteilung Access Token im Local Storage plus Refresh Token im HTTP-only Cookie ist der Aufbau aus dem Unterricht und ein verbreiteter Kompromiss.

8 Der komplette Ablauf in Home Control



Wichtig für das Frontend

- Bei einem 401 genau **einmal** refreshen und die ursprüngliche Anfrage wiederholen. Scheitert der Refresh (401), zurück zum Login.
- Immer nur **ein** Refresh gleichzeitig. React StrictMode ruft im Development Effekte doppelt auf. Zwei gleichzeitige Refreshs mit demselben Token würden die Reuse Detection auslösen und den Benutzer ausloggen.
- Bei 403 nicht refreshen: Ein neues Token ändert die Rolle nicht.

9 Der Code

Dateien

Datei	Aufgabe
Models/User.cs	Benutzer: Id, Email (eindeutig, klein geschrieben), PasswordHash, Role, CreatedAtUtc; Konstanten Roles.Admin/User
Models/RefreshToken.cs	TokenHash (eindeutig), UserId, CreatedAtUtc, ExpiresAtUtc, RevokedAtUtc
Data/ApplicationDbContext.cs	Tabellen Users und RefreshTokens, Indizes, Kaskade, Zeitpunkte als Unix-Millisekunden
Migrations/..._AuthentifizierungErgaenzt.cs	Legt beide Tabellen an (beim Start automatisch angewendet)
Auth/JwtOptions.cs	Issuer, Audience, SigningKey, AccessTokenMinutes, RefreshTokenDays
Auth/TokenService.cs	JWT erzeugen, Refresh Token erzeugen und hashen, Prüfregeln für die Middleware
Auth/AuthService.cs	Register, Login, Refresh mit Rotation und Reuse Detection, Logout
Endpoints/AuthEndpoints.cs	Die fünf Endpunkte, Eingaben prüfen, Cookie setzen und löschen, 401/403
Contracts/AuthContracts.cs	RegisterRequest, LoginRequest, UserResponse, LoginResponse
jwt-schluessel-einrichten.sh	Erzeugt den Signaturschlüssel und legt ihn in die User Secrets

Endpunkte

Methode und Pfad	Geschützt	Erfolg	Fehler
POST /api/auth/register	Erster Benutzer: nein, danach Admin	201	400, 401, 403
POST /api/auth/login	nein	200 + Cookie	400, 401
POST /api/auth/refresh	Refresh-Cookie	200 + neues Cookie	401
POST /api/auth/logout	nein	204	–
GET /api/auth/profile	Access Token	200	401

Middleware-Reihenfolge in Program.cs

```
app.UseExceptionHandler();   // 1. fängt alle Fehler ab (steht ganz vorne)
app.UseStatusCodePages();    // 2. leere Fehlerantworten (401, 403, 404) -> ProblemDetails
app.UseAuthentication();     // 3. Wer bist du?  Token lesen und prüfen -> HttpContext.User
app.UseAuthorization();      // 4. Was darfst du? RequireAuthorization, Rollen
// ... Migrationen, Scalar ...
app.MapAuthEndpoints();      // 5. Endpunkte
```

Authentication muss vor Authorization stehen: Ohne zu wissen, wer anfragt, lässt sich nicht entscheiden, was er darf. Beide stehen hinter dem Exception Handler, damit auch ihre Fehler als ProblemDetails ankommen.

JWT-Bearer einrichten

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.MapInboundClaims = false;   // "sub", "email", "role" so lassen
        options.TokenValidationParameters = TokenService.CreateValidationParameters(jwtOptions);
    });
builder.Services.AddAuthorization();

auth.MapGet("/profile", GetProfile).RequireAuthorization();   // ohne gültiges Token: 401
```

Konfiguration und Geheimnisse

Schlüssel	Wo	Hinweis
Jwt:Issuer, Jwt:Audience	appsettings.json	„HomeControl“, kein Geheimnis
Jwt:AccessTokenMinutes / RefreshTokenDays	appsettings.json	15 / 30
Jwt:SigningKey	User Secrets	Geheim! Wer ihn kennt, kann sich selbst Admin-Tokens ausstellen. Einrichten mit `./jwt-schluessel-einrichten.sh`

Fehlt der Schlüssel, startet Home Control im **Development** trotzdem, mit einem zufälligen Schlüssel nur für diesen Start und einer Warnung im Log. Tuya, Blink und der Rest laufen also weiter. In jeder anderen Umgebung startet die App ohne Schlüssel nicht, lieber gar nicht als unsicher.

Tests (HomeControl.Api.Tests)

Testklasse	Prüft
TokenServiceTests	Token gültig und enthält nur Id, E-Mail, Rolle · 15 Minuten Laufzeit · falscher Schlüssel abgelehnt · verändertes Token (User → Admin) abgelehnt · abgelaufenes Token abgelehnt · Refresh Token lang und zufällig · Hash gleichbleibend
AuthServiceTests	Erster Benutzer wird Admin, Passwort gehasht · zweiter Benutzer ohne Admin abgelehnt · Admin legt User an · doppelte E-Mail (andere Schreibweise) abgelehnt · Login ok · falsches Passwort · unbekannte E-Mail · Rotation · abgelaufenes Refresh Token · nach Logout widerrufen · unbekanntes Token · Reuse Detection

Die Tests laufen mit einer eigenen SQLite-Datenbank im Arbeitsspeicher und einer steuerbaren Uhr (FakeTimeProvider). Es gibt keine echten Anfragen an Tuya oder Blink.

10 Bekannte Grenzen

- **Access Token im Local Storage** ist bei XSS lesbar. Abgemildert durch 15 Minuten Laufzeit.
- **Access Token nicht widerrufbar.** Nach Logout oder Rollenwechsel gilt ein altes Access Token noch bis zu 15 Minuten.
- **Kein Schutz gegen Passwort-Raten** (Rate Limiting oder Sperre nach Fehlversuchen) bisher.
- **Zeitunterschied beim Login:** Bei unbekannter E-Mail wird kein Hash berechnet, die Antwort kommt etwas schneller. Theoretisch lässt sich daran erkennen, ob eine E-Mail existiert.
- **Lokal kein HTTPS.** Im echten Betrieb (auch auf dem iPad) ist HTTPS Pflicht, sonst gilt das Secure-Cookie nicht.

- **Bestehende Endpunkte noch offen.** Die Absicherung folgt schrittweise. Der n8n-Agent braucht vorher einen eigenen Zugang (API-Schlüssel).

11 Kurzantworten für Rückfragen

Warum speichert ihr das Passwort nicht verschlüsselt?
Weil wir es nie wieder im Klartext brauchen. Ein Hash lässt sich nicht zurückrechnen, ein verschlüsseltes Passwort schon, wenn jemand den Schlüssel erbeutet.

Was ist der Unterschied zwischen Salt und Signaturschlüssel?
Das Salt ist pro Passwort zufällig und darf neben dem Hash stehen. Es verhindert, dass gleiche Passwörter gleiche Hashes ergeben. Der Signaturschlüssel ist ein einziges Geheimnis des Servers, mit dem JWTs signiert werden.

Kann man ein JWT fälschen?
Lesen ja, ändern nein. Ohne den geheimen Schlüssel kann niemand eine passende Signatur erzeugen. Der Server lehnt jedes veränderte Token ab.

Warum braucht man überhaupt ein Refresh Token?
Damit das Access Token kurz leben kann (wenig Schaden bei Diebstahl), ohne dass man sich alle 15 Minuten neu einloggen muss.

Was bringt Rotation?
Jedes Refresh Token gilt nur einmal. Wird ein altes noch einmal benutzt, ist es wahrscheinlich gestohlen. Dann werden alle Sitzungen des Benutzers beendet.

Warum HTTP-only?
Damit JavaScript, auch eingeschleustes, das langlebige Refresh Token nicht lesen kann.

Was ist CSRF und wie schützt SameSite davor?
Eine fremde Seite lässt den Browser eine Anfrage an uns schicken, inklusive Cookies. Mit SameSite=Strict lässt der Browser das Cookie bei solchen Anfragen weg.

401 oder 403, wenn ein User einen Raum löschen will?
403: Er ist angemeldet (sonst wäre es 401), hat aber nicht die Rolle Admin.

Wo liegt der geheime Schlüssel?
In den .NET User Secrets, außerhalb des Projektordners und nie in Git. Später als Umgebungsvariable.

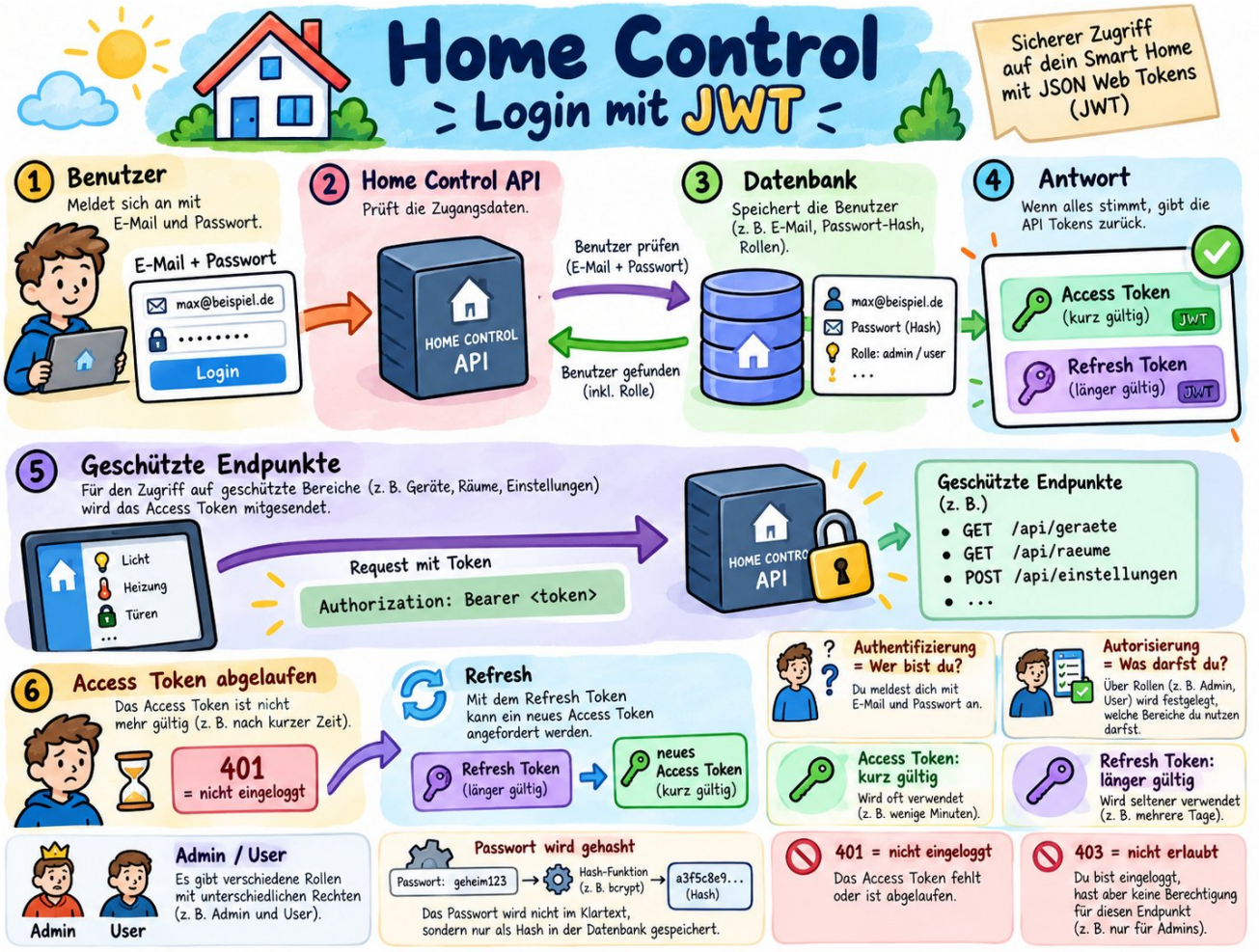
Warum steht im JWT keine Rolle aus der Datenbank, sondern eine Kopie?
Damit der Server bei jeder Anfrage nicht die Datenbank fragen muss. Der Preis: Eine geänderte Rolle wirkt erst mit dem nächsten Access Token, also nach spätestens 15 Minuten.

12 Merksätze

- Erst authentifizieren (Wer bist du?), dann autorisieren (Was darfst du?).
- 401 = nicht angemeldet · 403 = angemeldet, aber nicht berechtigt.
- Ein JWT ist signiert, nicht verschlüsselt: lesbar, aber nicht veränderbar.
- Access Token kurz (15 Min.), Refresh Token lang (30 Tage), widerrufbar und rotierend.
- Passwörter: langsamer Hash mit Salt. Nie Klartext, nie Verschlüsselung.
- HttpOnly schützt vor XSS-Diebstahl, SameSite schützt vor CSRF, Secure erzwingt HTTPS.
- Geheime Schlüssel gehören in User Secrets oder Umgebungsvariablen, nie in Git.
- UseAuthentication() vor UseAuthorization().

Anhang: Infografiken

Die Bilder zeigen das Prinzip. Abweichungen zu Home Control: Das Refresh Token ist kein JWT, sondern ein Zufallswert. Gehasht wird mit PBKDF2, nicht bcrypt. Nur das Refresh Token liegt im Cookie. Angemeldet wird mit E-Mail statt Benutzername. Geräte werden nicht gesteuert (Tuya und Blink nur lesend).



Authentifizierung & Autorisierung

Authentifizierung = Wer bist du?

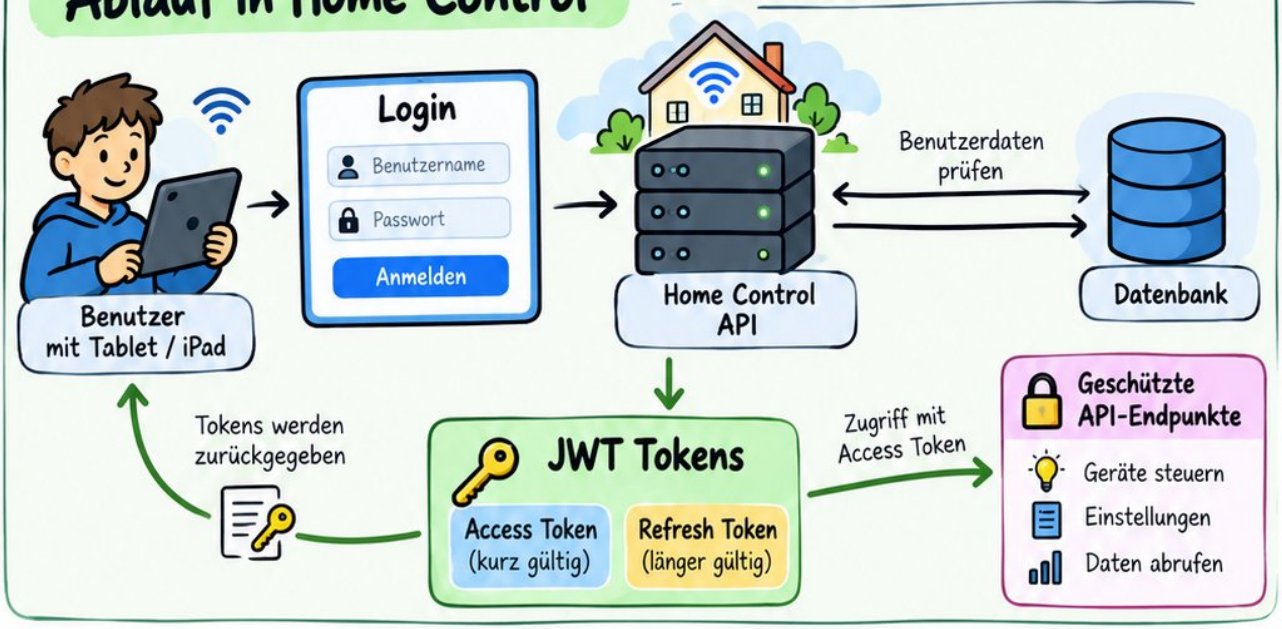
Hier wird geprüft, ob du wirklich der bist, für den du dich ausgibst.
(z. B. mit Benutzername und Passwort)

Autorisierung = Was darfst du?

Hier wird geprüft, welche Berechtigungen du hast und auf welche Funktionen oder Daten du zugreifen darfst.
(z. B. als Admin oder normaler User)

Ablauf in Home Control

Beispiel: Login und Zugriff auf geschützte Funktionen



JWT einfach erklärt

- **Access Token:** kurz gültig
  Wird für den Zugriff auf geschützte API-Endpunkte verwendet.
- **Refresh Token:** länger gültig
  Wird benutzt, um einen neuen Access Token zu erhalten.
- **Refresh-Token-Rotation:**
  Altes Token wird ersetzt. Das macht die Nutzung sicherer.

Rollen

Admin
- volle Berechtigungen
- Geräte verwalten
- Einstellungen ändern

User
- eingeschränkte Berechtigungen
- z. B. nur Geräte steuern
- keine Admin-Funktionen

Rollen steuern die Berechtigungen.

Wichtige Sicherheit

Passwort-Hashing
Passwörter werden nicht im Klartext gespeichert, sondern als sicherer Hash (z. B. bcrypt).

HTTP-only Cookie
Tokens werden in einem Cookie gespeichert, das nicht über JavaScript ausgelesen werden kann.

XSS (Cross-Site Scripting)
Schutz vor dem Einschleusen und Ausführen von schädlichem JavaScript.

CSRF (Cross-Site Request Forgery)
Schutz vor unerlaubten Anfragen, z. B. durch eine fremde Webseite.

Merke

- **401** = nicht authentifiziert
- **403** = keine Berechtigung

Erst authentifizieren, dann autorisieren.